FeCr-TWIN — Deployment guide (two GitHub repositories + Render)

Extractmet Private Limited · FeCr-TWIN v1.0

Architecture for remote viewing, simulation and advisory use:

```
Browser ──HTTPS/WSS──▶ Render Static Site (ferrochrome-saf-frontend)
   │
   └──HTTPS/WSS──▶ Render Web Service (ferrochrome-saf-backend, FastAPI) ──▶ Render Postgres
                                                                  └──▶ persistent disk (ML model
files)
Plant (optional, later): DCS/PLC ──read-only──▶ on-prem gateway (DMZ) ──outbound HTTPS
only──▶ backend
```

The cloud service is **never** in a control path. No component writes to plant controllers.

---

1. Prerequisites

Tool	Version used for this release
Git + a GitHub account (organisation recommended)	any current
Python	3.13 (3.12 works)
Node.js	22 LTS, npm 10
Render account	linked to GitHub

2. Unpack and run locally

Linux / macOS

```
unzip ferrochrome-saf-backend.zip && unzip ferrochrome-saf-frontend.zip
cd ferrochrome-saf-backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
cp .env.example .env            # edit SECRET_KEY etc. (development defaults work)
alembic upgrade head             # creates the SQLite dev database
uvicorn app.main:app --reload --port 8000
# new terminal
cd ../ferrochrome-saf-frontend
npm ci
cp .env.example .env             # VITE_API_BASE_URL=http://localhost:8000/api/v1
npm run dev                      # http://localhost:5173
```

Windows PowerShell

```
Expand-Archive ferrochrome-saf-backend.zip .; Expand-Archive ferrochrome-saf-
frontend.zip .
```

```
cd ferrochrome-saf-backend
py -3.13 -m venv .venv; .\.venv\Scripts\Activate.ps1
pip install -r requirements-dev.txt
Copy-Item .env.example .env
alembic upgrade head
uvicorn app.main:app --reload --port 8000
# new terminal
cd ..\ferrochrome-saf-frontend
npm ci; Copy-Item .env.example .env; npm run dev
```

Sign in at http://localhost:5173 as `engineer` / `extractmet-demo` (the `DEMO_PASSWORD`). Local Postgres instead of SQLite: `docker compose up -d db` then set `DATABASE_URL=postgresql://fecr:fecr@localhost:5432/fecr` and rerun `alembic upgrade head`; or run the whole backend with `docker compose up --build`.

Run the tests:

```
cd ferrochrome-saf-backend && python -m pytest -q tests && python
scripts/check_balances.py
cd ../ferrochrome-saf-frontend && npm test && npm run build
# browser e2e (backend on :8000, `npm run preview` on :4173)
npx playwright install chromium && FECR_WEB=http://localhost:4173 npx playwright test
```

3. Create the two GitHub repositories

Create two **private** repositories in your organisation: `ferrochrome-saf-backend` and `ferrochrome-saf-frontend` (no README/licence, they are in the archives).

```
cd ferrochrome-saf-backend
git init -b main && git add . && git commit -m "FeCr-TWIN backend v1.0"
git remote add origin https://github.com/<org>/ferrochrome-saf-backend.git
git push -u origin main

cd ../ferrochrome-saf-frontend
git init -b main && git add . && git commit -m "FeCr-TWIN frontend v1.0"
git remote add origin https://github.com/<org>/ferrochrome-saf-frontend.git
git push -u origin main
```

PowerShell is identical except for path separators. Both repositories carry a GitHub Actions workflow (`.github/workflows/ci.yml`). The frontend e2e job checks out the backend repo; add a fine-grained token with read access to it as the secret `BACKEND_REPO_TOKEN` (or delete the e2e job).

4. Deploy the backend first

Option A — Blueprint (recommended)

1. Render dashboard → **New → Blueprint** → select `ferrochrome-saf-backend`. Render reads `render.yaml` and proposes: web service `fecr-twin-api` (Python), Postgres `fecr-twin-db`, disk `fecr-models` (1 GB at `/var/data`).
2. Fill the variables marked `sync: false`:
   - `DEMO_PASSWORD` — a strong password (required; the app refuses the default in production).

- `ALLOWED_ORIGINS` — the frontend URL, e.g. `https://fecr-twin-desk.onrender.com` (set after step 5; a placeholder is fine now).

3. Apply. Build runs `pip install -r requirements.txt`; `preDeployCommand: alembic upgrade head` migrates the database; start command `uvicorn app.main:app --host 0.0.0.0 --port $PORT --proxy-headers`; health check `/health`.

Option B — manual web service

New → Web Service → repo → Runtime *Python 3* → Build `pip install -r requirements.txt` → Start `alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port $PORT --proxy-headers` (use this combined command on plans without pre-deploy commands) → Health check path `/health` → environment variables:

Variable	Value
`APP_ENV`	`production`
`SECRET_KEY`	long random string (Render "Generate")
`DATABASE_URL`	Internal Database URL of your Render Postgres (the app converts `postgres://` to the psycopg3 driver)
`ALLOWED_ORIGINS`	`https://<frontend>.onrender.com` (comma-separate several)
`DEMO_PASSWORD`	strong password for the seeded role accounts
`MODEL_DIR`	`/var/data/models` (attach a disk at `/var/data`)
`SPINUP_H`	`20`
`PYTHON_VERSION`	`3.13.5`

Plans: the free plan sleeps after inactivity (cold start 30–60 s plus model warm-up) and cannot attach a disk — acceptable for a quick look only. Use **Starter** or higher for demonstrations to customers, and a non-sleeping instance with backups for any pilot. Render free/sleeping services are **not** appropriate for time-critical plant use.

Verify the backend

```
        curl https://<backend>.onrender.com/health
        curl https://<backend>.onrender.com/ready
        curl -X POST https://<backend>.onrender.com/api/v1/auth/login -H "Content-Type:
application/json" \
             -d '{"username":"engineer","password":"<DEMO_PASSWORD>"}'
```

PowerShell:

```
        Invoke-RestMethod https://<backend>.onrender.com/health
        $t = (Invoke-RestMethod -Method Post -Uri
https://<backend>.onrender.com/api/v1/auth/login `
              -ContentType 'application/json' -Body '{"username":"engineer","password":"
<DEMO_PASSWORD>"}').access_token
        Invoke-RestMethod -Headers @{Authorization="Bearer $t"}
https://<backend>.onrender.com/api/v1/capabilities
```

`/ready` reports database status, active ML models (trained automatically from the shipped synthetic campaign on first start) and whether the warm-start cache is ready (about 20 s after boot). Interactive API docs: `https://<backend>.onrender.com/docs`.

5. Deploy the frontend

1. Render → **New → Static Site** (or Blueprint with the frontend `render.yaml`) → repo `ferrochrome-saf-frontend`.
2. Build command `npm ci && npm run build`; Publish directory `dist`.
3. Environment variables (build time; never secrets):
   - `VITE_API_BASE_URL=https://<backend>.onrender.com/api/v1`
   - `VITE_WS_BASE_URL=wss://<backend>.onrender.com/api/v1/ws/telemetry`
4. No SPA rewrite is needed (hash routing).
5. Copy the static-site URL into the backend `ALLOWED_ORIGINS` and redeploy the backend.

6. End-to-end checks

1. Open the static site, sign in, press **Start simulation**.
2. Header shows **Live stream** (WebSocket). If it shows *polling*, check `VITE_WS_BASE_URL` uses `wss://` and the backend URL; the desk keeps working over HTTP polling.
3. Browser developer console: no CORS or 4xx/5xx errors.
4. Visit every screen; 3D furnace should render (WebGL); the schematic should animate with values.
5. MPC → Compute recommendation (5–20 s) → accept with a reason → Data, tags and audit shows the entry.

7. Operations

- **Logs**: structured JSON on stdout (Render Logs) with request IDs, path, status, duration.
- **Observability**: `/health` (liveness), `/ready` (readiness incl. DB and models); add Render alerts.
- **Secrets**: only in Render environment variables; rotate `SECRET_KEY` to invalidate all tokens.
- **Backups**: enable Render Postgres point-in-time recovery; back up the model disk (or retrain from campaign).
- **Upgrades**: push to `main` → auto-deploy; Alembic migrations run before the new version starts.
- **Rollback**: Render → Deploys → *Rollback* to the previous deploy; database migrations should be backward-compatible (add-only) to allow this.
- **Restore**: create a Postgres from a backup/PITR point and point `DATABASE_URL` to it.
- **CI**: GitHub Actions run unit/integration/contract/security tests, balance checks, OpenAPI export and a dependency audit on every push.

8. Plant integration (pilot phase, not part of the cloud demo)

1. Install the on-prem gateway (Linux VM in the plant DMZ) with `asyncua` and the read-only adapter (`app/integrations/opcua_readonly_adapter.py`), a read-only OPC UA user and certificate, and an allow-list of node IDs mapped to twin tags (`/telemetry/mapping` CSV format).
2. The gateway pushes outbound only (HTTPS) to the backend; no inbound ports are opened in the plant.
3. Keep the twin in **online shadow** mode (read-only); recommendations remain advisory.
4. Follow the commissioning checklist in `OPERATOR_MANUAL.md` and the conditions in `MPC_SAFETY_CASE.md`.

9. Troubleshooting

Symptom	Cause / fix
Browser: *CORS policy* error	`ALLOWED_ORIGINS` must equal the frontend origin exactly (scheme + host, no trailing slash); redeploy backend
Backend unreachable in sign-in	wrong `VITE_API_BASE_URL` (rebuild frontend after changing it) or backend sleeping (first request wakes it)
Service fails: `SECRET_KEY must be set` / `DEMO_PASSWORD must be changed`	production guard — set the variables
Port binding error	start command must use `--port $PORT` and host `0.0.0.0`
WebSocket stays *polling*	use `wss://` with HTTPS; check `VITE_WS_BASE_URL`; proxies must allow upgrades; uvicorn needs the `websockets` package (pinned)
`relation ... does not exist`	migrations not run — `alembic upgrade head` (pre-deploy or start command)
Postgres URL scheme error	use the Internal Database URL; `postgres://` is converted automatically
First session of a new MVA size is slow	warm-start spin-up (~10–20 s); cached afterwards; lower `SPINUP_H` if needed
MPC request slow / timed out	6 h horizon identification ~5–20 s on small instances; the gate blocks on timeout (safe); use a larger plan
ML models missing in `/ready`	the disk is not mounted or `MODEL_DIR` not writable; check logs for bootstrap errors
Vite build: `Cannot find module three`	run `npm ci` (lockfile committed); Node 22
3-D view shows text fallback	browser/GPU without WebGL — use the schematic and text readout
OIDC callback errors (if you integrate an IdP)	redirect URI must match the static-site URL; tokens validated server-side in `core/security.py`
Sessions vanish	in-memory by design; free plan sleep or redeploy restarts them — recreate (scenarios/audit persist in DB)